

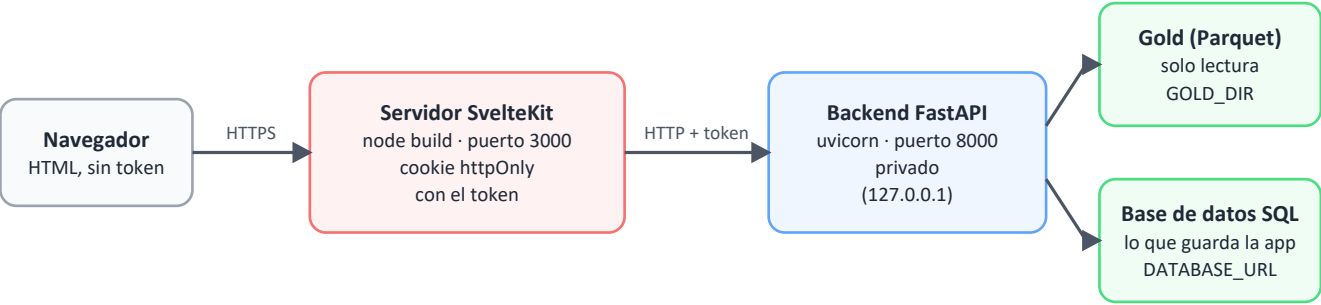
Web de Key Perú: estado de la integración y guía de despliegue

Para quien despliega la web (el responsable del frontend) y para Aldair, que la conectó con el backend.
Estado de la rama `feat/front-integration` al 5 de octubre de 2026.

1. Estado actual

- **La web funciona de punta a punta.** Incluye inicio y cierre de sesión, un menú por rol y el chat del cliente en todos sus estados: código de verificación, confirmación, transferencia a un asesor, conversación terminada, sesión expirada y retomar. Las páginas del cliente usan datos reales del gold, y el agente tiene una bandeja de casos.
- **Lo que se agregó al backend para la web.** `GET /cliente/*` lee las tarjetas, transacciones, conversaciones y casos del cliente que inició sesión (regla POL-PII-10). `POST /autenticacion/otp-demo` envía el código de verificación como un SMS simulado, solo en desarrollo (POL-AUTH-14). Política `cards-synthetic-0.9`, contrato de API `chat-api-0.2`.
- **Verificado el 5 de octubre de 2026.** Pasaron: 147 pruebas del backend, `svelte-check` sin errores ni advertencias, 57 comprobaciones de punta a punta a través de las rutas de SvelteKit, una prueba del chat en un navegador sin interfaz y el build de producción servido con `node build`.
- **Rama.** Todo está en `feat/front-integration`, junto con el merge de `main`. Cuando se escribió esto aún no tenía commit: despliega desde esa rama cuando Aldair la suba, o desde `main` cuando se fusione.

2. Cómo se conectan las piezas



- El navegador solo habla con el servidor de SvelteKit. SvelteKit llama a FastAPI con el token del usuario, que vive en una cookie httpOnly que el código del navegador no puede leer.
- FastAPI no necesita ser público, porque solo lo llama el servidor de SvelteKit. Haz que escuche solo en `127.0.0.1` en el mismo host, o déjalo en una red privada.
- Los datos del banco son de solo lectura: FastAPI copia el gold (Parquet) a memoria en la primera consulta. Todo lo que escribe la app va a la base SQL: usuarios, sesiones, conversaciones, casos, registro de auditoría y bloqueos de tarjetas.

3. Roles

Rol	Quién es	Qué ve en la web	Qué datos ve
cliente	Un cliente del banco. Cada usuario de prueba está ligado a un <code>customer_id</code> del gold.	Resumen, Consultas (el chat con el asistente), Mis tarjetas, Transacciones y Mis casos. Puede bloquear su propia tarjeta con un código de verificación y un “Sí” explícito.	Solo los suyos.
agente	El asesor humano que recibe las transferencias.	Bandeja de casos: cada caso, su expediente (cliente, IDs internos, hechos verificados, preguntas abiertas) y el registro de auditoría de su conversación, con la verificación de la cadena de hashes.	Todos los casos.

Rol	Quién es	Qué ve en la web	Qué datos ve
jurado	Un evaluador.	Un aviso. La consola de prueba (buscar clientes, abrir una sesión de prueba y emitir códigos) por ahora solo está en el CLI.	Cualquier cliente, mediante una sesión de prueba identificada como tal.

Para ver los dos lados, entra como cliente y escribe “Quiero hablar con un asesor” en Consultas. Anota la referencia **CASE-...**, luego entra como **agente@keyperu.example** y abre ese caso.

4. Usuarios de prueba

Los crea el script de seed. Todos comparten la **SEED_PASSWORD** del backend, que se comparte en privado y nunca va a git. Todos los correos terminan en **@clientes.keyperu.example**, llevan el ID del cliente en minúsculas, y la barra superior lo muestra después de iniciar sesión. El asistente y las páginas siempre usan al cliente de la sesión iniciada: un ID de cliente escrito en el chat se ignora.

Usuario	Escenario golden	Tarjetas (últimos 4, tipo, estado)	Mov. en 30 días	Productos con saldo
cli-5b9vscp2gsml	1 (es): cargo rechazado	6898 débito activa; 6873 crédito activa; 0727 crédito bloqueada	2	6873, 0727
cli-jpk27b33sv65	2 (pt): bloquear una tarjeta perdida	8407 crédito activa	0	8407
cli-petsg0sjc2g2	3 (es): varias tarjetas, “mi tarjeta”	9130, 6000, 3968 crédito activas	1	9130, 6000, 3968
cli-fw98ujsyflwx	4 (pt): pedido fuera de alcance (préstamo)	ninguna	0	ninguno
cli-ayahyqeg16bz	5 (es): reclamar un cargo	1883 débito activa; 4950 crédito activa	1	4950
cli-an7kxgr09tb2	6 y 12 (pt): por qué se bloqueó; saldos	7858, 2960 crédito activas; 2952 crédito bloqueada	0	7858, 2960, 2952; ahorro 2700
cli-ghrmpxt32bkk	7 (es): tarjeta de otro cliente	2771 crédito activa; 7167 débito activa	1	2771
cli-ehvv6yj6s15w	8 (es): intento de manipulación (prompt injection)	8283, 7131 débito activas; 0245 crédito activa; 7663 crédito bloqueada	1	7663, 0245; ahorro 7562 (bloqueada)
cli-ld6qvncstr43	9 (es): la sesión expira a mitad del flujo	4214 débito activa	0	ninguno
cli-jas4v4u7h60h	10 (pt): falla de una herramienta al bloquear	7042, 0044 crédito activas	2	7042, 0044; ahorro 7645, 0614
cli-sqjocedjjncz	11 (es): saldo de tarjeta de crédito	5070 crédito activa; 5419 débito cerrada	0	5070; ahorro 1317

“Mov. en 30 días” cuenta las transacciones de tarjeta del 19 de mayo al 18 de junio de 2026, según el reloj del banco que fija **RELOJ_SIMULADO**. Los productos con saldo son tarjetas de crédito y cuentas de ahorro que no están cerradas. La persona 4 no tiene tarjetas a propósito, así que sus páginas salen vacías. El seed también crea **agente@keyperu.example** y **jurado@keyperu.example**. El gold de demo tiene las mismas filas que el gold completo para estos 11 clientes.

5. Dónde están los datos

Almacén	Qué contiene	Se configura con
Gold, archivos Parquet (solo lectura)	<code>customers</code> , <code>cards</code> , <code>card_transactions</code> , <code>balance_products</code>	<code>GOLD_DIR</code> : la carpeta con los <code>.parquet</code> , o su carpeta padre <code>03_primary</code>
Base de datos SQL	<code>usuarios</code> , <code>sesiones_identidad</code> , <code>conversaciones</code> , <code>mensajes_chat</code> , <code>casos</code> , <code>eventos_auditoria</code> , <code>eventos_estado_tarjeta</code> (bloqueos de tarjetas) y las tablas del <code>step-up</code>	<code>DATABASE_URL</code>

- Un bloqueo nunca cambia el Parquet. Agrega una fila a `eventos_estado_tarjeta`, y el estado que se muestra es el del gold más esas filas.
- Las fechas de las transacciones siguen el reloj del banco (`RELOJ_SIMULADO`, 18 de junio de 2026 a las 10:00). El saldo se informa “según los datos” de la fecha en que se cargó el gold (`gold_loaded_at`: 3 de octubre de 2026 para el gold actual), no según el reloj del banco.
- Para reiniciar la demo (bloqueos, casos y conversaciones), recrea la base SQL: borra el archivo SQLite y vuelve a correr el script de seed. El gold no cambia.

Revisar un cliente a mano

Desde `backend/`, abre `.venv/Scripts/python` y pega esto. Cambia `cid` por el cliente que inició sesión:

```
import duckdb, sqlite3
g = '../ml/data/03_primary/gold'          # tu GOLD_DIR
cid = 'CLI-5B9VSCP2GSML'

duckdb.sql(f"select card_id, last4, product_type, status "
           f"from '{g}/cards.parquet' where customer_id = '{cid}'").show()
duckdb.sql(f"select transaction_datetime, transaction_type, amount, currency, merchant_name, status "
           f"from '{g}/card_transactions.parquet' where customer_id = '{cid}' "
           f"and transaction_datetime >= '2026-05-19' order by transaction_datetime desc").show()
duckdb.sql(f"select kind, last4, status, currency, current_balance, credit_limit "
           f"from '{g}/balance_products.parquet' where customer_id = '{cid}'").show()

c = sqlite3.connect('key_peru.db')          # tu DATABASE_URL
print(c.execute("select card_id, nuevo_estado, hora_evento "
               "from eventos_estado_tarjeta").fetchall())          # bloqueos hechos desde la app
print(c.execute("select case_id, priority, created_at from casos "
               "where customer_id = ?", (cid,)).fetchall())          # casos del cliente
```

La web muestra solo los últimos 4 dígitos. El `card_id` de la primera consulta es el que aparece en `eventos_estado_tarjeta` después de un bloqueo.

6. Despliegue

6.1 Qué va al servidor

Elemento	Cómo	Notas
El repositorio	<code>git clone</code> , rama <code>feat/front-integration</code> (o <code>main</code> cuando se fusione)	Despliega el repositorio completo, no solo <code>frontend/</code> y <code>backend/</code> . El backend importa <code>ml/src/banking_cs/nlu</code> al arrancar y carga el modelo de intenciones desde <code>ml/artifacts/nlu</code> (1.5 MB, está en git).
Datos del gold	No están en git. Copia <code>ml/data/03_primary/gold/</code> (79 MB, todos los clientes), o descomprime <code>ml/data/demo_gold.zip</code> (211 clientes, incluidos los 11 de prueba).	Pídele los archivos a Aldair y apunta <code>GOLD_DIR</code> a esa carpeta.
Secretos	Variables de entorno del host (6.2)	Nunca se suben a git, y nunca se copia el <code>backend/.env</code> de una laptop.

Elemento	Cómo	Notas
Base de datos	Un archivo SQLite que crea el script de seed, o una URL de Postgres	Para la demo basta un archivo SQLite.

No subas `.venv`, `node_modules` ni archivos `.env`: instala y configura en el servidor.

6.2 Backend (FastAPI, Python 3.11)

Corre todos los comandos desde `backend/`: los módulos usan imports planos y el `.env` se lee desde la carpeta de trabajo.

```
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
python -m pip install --upgrade pip # el pip de Python 3.11.0 no instala scikit-learn 1.9
pip install -r requirements.txt
# define las variables de abajo (entorno del host, o un backend/.env escrito en el servidor)
python -m scripts.sembrar_usuarios # crea las tablas y los usuarios de prueba; se puede repetir
uvicorn main:app --host 127.0.0.1 --port 8000 # un solo proceso, sin --reload
```

Variable	Valor para la demo	Por qué
ENV	<code>development</code> o <code>production</code> : ver el recuadro de abajo	Activa el registro abierto, la creación de tablas al arrancar y el SMS simulado.
DATABASE_URL	<code>sqlite:///./key_peru.db</code>	Obligatoria.
SECRET_KEY, AUDIT_KEY	Un valor aleatorio nuevo para cada una, en cada despliegue	Genéralo con <code>python -c "import secrets; print(secrets.token_urlsafe(48))"</code> . El backend no arranca sin ellas.
CARD_HASH_KEY	La clave del equipo, la misma que usó el pipeline del gold	Obligatoria para arrancar. Solo la verificación de un número de tarjeta completo escrito en el chat necesita la clave real.
SEED_PASSWORD	Fuerte, de 12 caracteres o más, solo para el jurado	Todos los usuarios de prueba la reciben. Si cambia, al volver a correr el seed se actualizan sus contraseñas.
GOLD_DIR	Ruta de la carpeta del gold en el servidor	Sin gold, las herramientas del banco fallan.
RELOJ_SIMULADO	<code>2026-06-18T10:00:00</code>	Sin ella el banco usa la fecha de hoy, y el gold no tiene movimientos recientes.
LLM_MODE	<code>mock</code> (por defecto)	<code>openai</code> además necesita <code>OPENAI_API_KEY</code> , y que <code>LLM_MODEL</code> esté en <code>LLM_ALLOWED_MODELS</code> .
ML_SRC_DIR, NLU_ARTIFACT_DIR	Sin definir	Solo si <code>ml/</code> no está junto a <code>backend/</code> .

Decisión del equipo: ENV en el servidor público

Con `ENV=production`, el registro abierto queda cerrado y las tablas no se crean al arrancar (las crea el script de seed). También se apaga el SMS simulado, así que en la web un bloqueo de tarjeta no puede recibir su código: solo el CLI del jurado puede emitirlo.

Con `ENV=development`, el bloqueo completo funciona en la web, pero cualquiera con la URL puede registrar un usuario. Ese usuario no tiene datos del banco y no puede chatear.

Hoy el README recomienda `production`. Si el jurado debe probar el bloqueo en la web, usen `development` y compartan la URL solo con el jurado.

Corre un solo proceso del backend. El límite de búsquedas y la copia del gold en memoria viven en ese proceso, y SQLite funciona mejor con un solo escritor.

6.3 Frontend (SvelteKit con adapter-node, Node 22 y pnpm 10)

```
cd frontend
pnpm install --frozen-lockfile
pnpm build # genera frontend/build/
PORT=3000 HOST=127.0.0.1 ORIGIN=https://<dominio público> \
  BACKEND_URL=http://127.0.0.1:8000 node build
```

Variable	Valor	Por qué
BACKEND_URL	La URL de FastAPI vista desde este servidor, por ejemplo <code>http://127.0.0.1:8000</code>	Se lee al ejecutar. El nombre antiguo <code>API_BCKD_8</code> todavía funciona.
ORIGIN	La URL pública, por ejemplo <code>https://keyperu.example</code>	Obligatoria detrás de un proxy. Sin ella, los formularios como “Cerrar sesión” fallan con 403 “Cross-site POST form submissions are forbidden” (comprobado en el build de producción).
PORT, HOST	Por defecto <code>3000</code> y <code>0.0.0.0</code>	Dónde escucha <code>node build</code> .

Tres cosas que rompen un despliegue

HTTPS es obligatorio en el dominio público. La cookie `token` es `Secure` fuera de localhost, así que por http el navegador la descarta y cada inicio de sesión vuelve a `/login`.

`node build` no lee archivos `.env`. Define las variables en el host o en el servicio, o corre `node --env-file=.env build`.

Corre `node build` desde `frontend/`, después de `pnpm install`. `d3-shape` se carga desde `node_modules` al ejecutar, así que la carpeta `build/` sola no basta.

Detrás de un proxy inverso (nginx, Caddy o el del host), termina el TLS ahí y reenvía al puerto 3000. Pon un tiempo de espera de lectura de al menos 140 segundos: con `LLM_MODE=openai`, un turno del chat puede esperar hasta 135 segundos al modelo.

6.4 Prueba rápida después de desplegar

- 1 En el servidor: `curl http://127.0.0.1:8000/` responde `{"Hello": "Backend de Key Peru"}`.
- 2 Abre `https://<dominio público>/login` e inicia sesión como `cli-5b9vscp2gsml@clientes.keyperu.example`. El Resumen muestra 3 tarjetas, 2 activas.
- 3 En Transacciones, la compra en Uber del 6 de junio de 2026 aparece como Rechazada.
- 4 En Consultas, pregunta “¿Cuál es mi saldo?”. El asistente pregunta cuál tarjeta: la 6873 o la 0727.
- 5 Solo con `ENV=development`: bloquea la 6873 desde Mis tarjetas. Pulsa “Recibir el código por SMS simulado” y luego “Sí”; la tarjeta pasa a Bloqueada. El bloqueo queda en la base hasta que se recree (sección 5).
- 6 En Consultas, escribe “Quiero hablar con un asesor” y anota la referencia `CASE-....`
- 7 Cierra sesión y entra como `agente@keyperu.example`: el caso aparece en la bandeja y muestra “Cadena de hashes verificada”.

7. Qué cambió en el código del frontend

Para quien hizo las pantallas: el diseño y los estilos siguen siendo los tuyos; estos son los cambios que hizo falta para la integración.

- **Sin datos de ejemplo.** Cada página carga datos reales en un `+page.server.ts` nuevo, con las funciones de `lib/server/backend.ts`. Ante un 401, `leerBackend` manda a `/login?volver=<página>`.
- **Chat** (`app/consultas`): la lógica es nueva y envía a `routes/api/chat/[accion]` (`sesiones`, `mensaje`, `codigo`). Su `<main>` interno pasó a ser un `<div>`, porque el layout ya tiene un `<main>`.
- **Menú y roles.** La barra lateral muestra solo las secciones del rol y tiene “Cerrar sesión” (`POST /logout`). `lib/data/roles.ts` decide qué páginas puede abrir cada rol, y `hooks.server.ts` lo aplica.

- **Páginas.** Los gráficos reciben sus datos por prop (**datos**). Mis tarjetas ya no muestra logo de Visa o Mastercard, porque el gold no tiene la red de la tarjeta. Transacciones guarda el periodo y la tarjeta en la URL (**?dias=**, **?tarjeta=**).
- **Inicio de sesión.** La ruta de login ya no le envía el token al navegador. La cookie dura 60 minutos (antes, 7 días) y, después de iniciar sesión, **?volver=** regresa a la página anterior.
- **Archivos nuevos.** **lib/types.ts**, **lib/utils/formato.ts**, **lib/data/roles.ts**, **routes/logout**, **app/bandeja** (bandeja y página del caso) y **app/+error.svelte**.
- **CSS.** **section p** y **section a** en **layout.css** están fuera de las capas de Tailwind, así que se imponen a las utilidades. Los colores de texto dentro de una sección necesitan **!**, como en **text-white!**. El layout raíz ya no enlaza **/favicon.ico**: el archivo no existía y daba un 404 en cada página.

8. Límites conocidos

- La consola del jurado solo existe en el CLI.
- Las lecturas del portal (**/cliente/***) no son turnos del chat, así que no quedan en el registro de auditoría.
- El SMS simulado solo funciona con **ENV=development** (sección 6.2).
- Un solo proceso del backend a la vez (sección 6.2).

Referencias

- Contrato de la API: **docs/contracts/chat_api.md** (las secciones 2 y 4.7 cubren la cookie y el portal).
- Ejecución local: **README.md**, “Run the agent locally”. Decisiones D-44 a D-47 en **docs/decisions_log.md**.
- Reglas: **docs/policy_cards.md** (**cards-synthetic-0.9**), POL-AUTH-14 y POL-PII-10.